

LLM 预训练之数据清洗

Data Cleaning • 深度研究手册

从“删垃圾”到“塑造训练分布”：规范化、过滤、质量评分、安全清洗、去污染与实验驱动阈值设计

研究快照：2026-08-13

核心资料：FineWeb • DCLM • Dolma • Nemotron-CC • FinerWeb • C4 • RedPajama • ROOTS

核心判断 数据清洗不是“把脏文本删除掉”这么简单。它本质上是在定义 $P_{\text{train}}(x)$ ：哪些语言、风格、知识密度、结构、风险类型和表达方式可以进入模型参数，以及以什么比例进入。

阅读地图

本手册把 Data Cleaning 与相邻环节明确分离，并用公开研究中的可复现实验结果解释每一类清洗动作为什么存在、怎样做、怎样验证。

章节	核心问题	你应带走的 Know-how
1. 边界与目标	Cleaning 到底包括什么？	把 parsing / cleaning / dedup / mixing 分离，避免指标混淆
2. 规范化	哪些是“修复”而不是“过滤”？	Unicode、空白、控制字符、格式残留要可逆、可观测
3. 语言清洗	Language ID 为什么不是简单分类？	阈值、代码混合、方言偏差、多语言路由
4. 启发式过滤	规则为什么仍有价值？	低成本、高吞吐；但阈值必须用 ablation 选
5. 模型式质量过滤	什么是 quality classifier？	参考正负样本比模型复杂度更重要
6. 粒度	删文档还是删行？	文档级便宜，行级精准；分层级联最实用
7. 安全/隐私	PII、毒性、敏感内容怎么处理？	风险目标与能力目标存在 trade-off
8. 去污染	Benchmark overlap 如何处理？	检测、报告、移除要独立于一般质量过滤
9. Horizon	为什么短训和长训需要不同清洗？	高纯度 vs 独特 token 数量的张力
10. 实验体系	怎么证明清洗有效？	固定模型/算力/Token，训练 proxy model 做 ablation
11. Production Blueprint	线上怎么落地？	Tag-first、reversible、版本化、分桶、观测与回溯

阅读方式 如果你做的是 LLM 数据平台，优先读第 1、4、5、9、10、11 章；如果你做 multilingual / safety，再重点读第 3、7 章。

1. 数据清洗的边界：先把概念切干净

预训练数据工程中，“清洗”经常被当成一个大筐：正文抽取、语言识别、去重、质量评分、PII、去污染都被统称为 cleaning。工程上这样做会导致两个问题：一是故障归因困难，二是无法对单个干预做严格 ablation。FineWeb 与 DCLM 都强调用固定训练配方隔离数据干预的因果效果。[1][2]

环节	主要输入	核心动作	输出	不要混淆
Parsing / 解析	HTML、PDF、Repo	恢复正文与结构	Canonical document	解析失败不是低质量
Cleaning / 清洗	Canonical document	规范化、语言/质量/风险过滤	Tagged / filtered documents	清洗不等于去重
Dedup / 去重	清洗后文本	Exact/Fuzzy/Substr/Semantic	Unique-ish corpus	重复性不是“质量”本身
Decontam / 去污染	训练语料+评测集	检测 benchmark overlap	Decontaminated corpus	必须单独报告

环节	主要输入	核心动作	输出	不要混淆
Mixing / 混合	多个数据桶	采样/上采样/配比	Training mixture	混合是分布设计，不是清洗

Know-why Cleaning 的真实目标不是“文本看起来干净”，而是提高单位训练 token 的有效学习信号，同时控制风险、偏差与分布损失。

推荐的逻辑顺序（具体工程可并行化）

Parsed Doc	→	Normalize	→	Tag	→	Score	→	Mask / Filter	→	Quality Buckets	→	Dedup	→	Mix
------------	---	-----------	---	-----	---	-------	---	---------------	---	-----------------	---	-------	---	-----

1.1 Cleaning 的四个目标函数

目标	希望优化什么	常见代理指标	潜在副作用
训练效率	同样 FLOPs 学到更多	下游 benchmark、loss 收敛速度	过拟合“高质量”风格
覆盖与多样性	保留足够独特 token	unique tokens、domain/lang coverage	噪声上升
风险治理	降低隐私/有害输出风险	PII 命中率、toxicity eval	误删少数群体/非正式语言
评测可信度	降低 benchmark contamination	overlap rate、contamination report	过度匹配导致误删正常文本

2. 规范化：先修复表示，再谈“好坏”

规范化（normalization）是数据清洗中最基础、也最容易被误做成“破坏性清洗”的环节。它的原则是：修复编码与格式表示差异，但尽量不改变语义和结构。

问题	典型现象	推荐处理	为何重要	风险
Unicode 规范	全角/组合字符/兼容字符差异	按语言需求采用 NFC/NFKC，并记录版本	同形文本统一，降低伪重复	NFKC 可能改变某些语义符号
不可见控制字符	零宽字符、BOM、异常控制码	白名单/黑名单 + 统计	避免 tokenizer 噪声与注入式伪文本	某些脚本合法使用零宽字符
空白与换行	大量空行、制表符混乱	折叠连续空白，但保留段落边界	减少格式噪声	过度折叠破坏代码/表格
HTML 残留	 、tag 片段、导航碎片	回到 parser 修复；清洗层仅做低风险残留处理	避免把 parser failure 当质量问题	直接 regex 清 HTML 容易误删正文
乱码/编码损坏	◆、mojibake	检测并标记，必要时 quarantine	乱码是高噪声监督信号	自动“猜修复”可能制造文本
异常字符比例	符号、数字、emoji 过密	统计为 tag，不直接一刀切	为后续规则/模型提供信号	不同领域分布差异极大

工程原则 Normalization 适合“可逆或低风险转换”；Quality Filtering 适合“选择”。不要把不可逆删除藏在 normalization 中。

3. Language ID：它是分布路由器，不只是英语过滤器

Dolma 使用 fastText 语言识别并采用相对宽松的英语阈值，以降低语言检测器对少数化英语方言的潜在偏差；FineWeb 的 base filtering 则保留 fastText 英语分数 ≥ 0.65 的样本。[1][3] 这说明 Language ID 阈值必须与目标语言覆盖和误杀成本一起设计。

设计点	简单做法	更稳健做法	为什么
文档级 vs 段落级	整篇一个语言	文档主语言 + 段落/跨度标签	网页常混合导航、引用和代码
阈值	固定 0.5/0.65	按语言资源量与误杀成本校准	低资源语言分类器更不稳定
Code-switch	判定为“非目标”	保留混合率与脚本特征	现实语言和技术文本常混合
未知/低置信度	直接删除	进入 unknown bucket / 二级模型	避免系统性丢失方言/稀有脚本
多语言预训练	一个统一 classifier	语言家族/脚本分层 + per-language quality	英语经验不能直接搬到所有语言

2025 年的多语言模型式数据选择研究在 FineWeb-2 多语种数据上发现，模型式过滤可以显著提升 token 效率，并强调不同语言家族/资源水平下需要验证泛化，而不能把英文清洗规则原样复制。[12]

Know-how Language ID 最好输出 score / language / script / mixed_ratio 等标签，而不是只输出 keep/drop；后续 mixing 才能利用这些信息。

4. 启发式过滤：低成本、高吞吐，但必须做消融实验

C4、MassiveText/Gopher、RefinedWeb、Dolma、FineWeb 都使用了启发式规则。FineWeb 的价值在于：它没有默认“历史规则都有效”，而是把规则当候选特征，通过小模型预训练 ablation 选择真正带来收益的阈值。[1]

信号类别	例子	想排除什么	误杀风险	建议
长度	文档/行太短	菜单、碎片、标签	标题、诗歌、问答	作为弱信号组合使用
终止标点比例	行是否以标点结束	列表/导航/破碎句	聊天、标题、代码	用文档比例而非逐行硬删
短行比例	<30 字符行占比	菜单、表格残片	诗歌/列表/对话	语言与领域校准
重复行比例	重复行字符占比	模板、SEO、页脚	合法重复结构	与 dedup 指标分开
字符组成	字母/数字/符号比例	乱码、数据 dump	公式、代码、财务表	按 domain 路由

信号类别	例子	想排除什么	误杀风险	建议
特殊短语	lorem ipsum、cookie policy	占位/策略模板	讨论这些短语的正文	低风险规则可硬过滤
URL/域规则	成人/恶意 URL blocklist	高风险站点	误伤域名/上下文	记录规则版本与命中原因

4.1 FineWeb：从 50+ 指标里筛出有效规则

FineWeb 先收集 50 多个文档统计特征，比较“相对高质量”和“相对低质量”数据的分布，再选择阈值并训练 1.82B 模型做 28B-token ablation。[1] 这套方法比“人工觉得某条规则合理”更可靠。

FineWeb 规则/决策	阈值或结论	观测结果/理由
英语过滤	fastText English score ≥ 0.65	作为 base filtering 之一
C4 terminal punctuation	逐行终止标点规则会移除约 30% token	单独收益大，但过于激进，因此最终未照搬
行终止标点比例	≤ 0.12 时过滤文档	较少删除，捕获破碎/列表型页面
重复行字符比例	≥ 0.10 时过滤文档	针对模板/重复内容
短行比例	长度 <30 字符的行占比 ≥ 0.67 时过滤	捕获导航/菜单型文本
三条新规则组合	约移除 22% token	28B-token ablation 中 aggregate score 约提升 1%

Know-why 规则过滤的真正资产不是 regex，而是“统计特征 → 候选阈值 → 训练 ablation → 保留收益/成本曲线”的实验方法。

5. 模型式质量过滤：Quality 是“相对于参考分布”的判别问题

DCLM 系统比较 PageRank、Semantic Dedup、embedding 线性分类器、AskLLM、perplexity、top-k logits 与 fastText 二分类器，结果发现 fastText-based filtering 最强。[2] 更反直觉的是：分类器的正样本构造与阈值，往往比“模型有多大”更关键。

方法	核心信号	优点	主要问题	DCLM 结论
Perplexity	小 LM 对文本的困惑度	简单、可解释	偏向训练 LM 熟悉的风格	不是最佳
Embedding classifier	语义向量分类	可表达主题/风格	成本较高、参考集敏感	不及 fastText
AskLLM	LLM 直接判断是否有帮助	语义灵活	昂贵、尺度化困难	不及 fastText
PageRank	网页链接重要性	外部结构信号	对无链接高质文本不友好	不及 fastText
fastText classifier	n-gram/词面判别	极快、易扩展	高度依赖正负样本定义	DCLM 最佳

DCLM 的一个重要 ablation：用 OpenHermes 2.5 + ELI5 作为“高质量”正样本，比传统 Wikipedia / Books / OpenWebText 类参考数据更有效；较严格的 top-10% 阈值也优于 top-15%/20%。[2] 这说明所谓 quality classifier 实际上在回答：“这段文本像不像我希望模型未来更擅长的目标分布？”

关键风险 如果正样本全部来自“教材式、解释式、标准英语”，模型过滤器会把这种风格当作 quality 本身，从而压缩语言多样性。Quality score 不是客观真理，而是一个可学习的价值函数。

5.1 评分而不是立刻删除

推荐：把清洗变成打标签/分桶，而不是单次不可逆删除

Doc	→	Heuristic tags	→	Quality model score	→	Risk scores	→	Bucket	→	Sampling policy
-----	---	----------------	---	---------------------	---	-------------	---	--------	---	-----------------

输出字段	示例	后续用途
quality_score	0.00-1.00	按阶段设置不同保留阈值
quality_bucket	A/B/C/D	mixing 时差异采样
reason_codes	short_lines, nav_like, spam	可解释与人工抽检
lang_score	en=0.82	低置信度二次路由
risk_scores	pii/toxic/adult	风险策略独立控制
parser_health	ok/degraded	避免把解析问题当质量问题
source/domain	news/forum/blog/...	做 coverage 监控

6. 清洗粒度：Document / Paragraph / Line / Span

传统管线多使用文档级过滤，因为吞吐最高；但一个 2,000 行网页只要有 20 行页脚、导航或版权信息，直接删整篇就损失大量正文。FinerWeb-10BT 的 2025/2026 工作专门验证了 LLM 标注 + 小分类器的行级清洗路径。[5]

粒度	优点	缺点	适合对象
Document	最便宜；规则简单；易分片	误删代价最高	严重乱码、极短、明显 spam
Paragraph	精度与成本平衡	段落边界依赖 parser	页脚、模板、引用段
Line	能保留“脏文档里的好正文”	推理/IO 成本高	导航、版权、元数据、格式残留
Span/Character	最精确	容易破坏语义；成本高	PII mask、少量格式噪声

6.1 FinerWeb 的可迁移做法

FinerWeb 先让 GPT-4o mini 对 20,000 个 FineWeb 文档逐行生成动态低质量标签，再把标签归并为 9 类，最后训练 DeBERTa-v3-base 扩展到 10B-token 数据。[5] 其工程启发不是“必须使用某个模型”，而是“昂贵 LLM 用于建立 taxonomy / teacher labels，小 encoder 用于大规模执行”。

FinerWeb 观测	数值/结论	工程含义
初始逐行标注	328,472 行；最终约 14% 被视为低质量	启发式清洗后的 FineWeb 仍有可识别残留噪声
9 类低质量	格式/错误、引用、spam、联系信息、导航、元数据、法律、冒犯等	quality 不是一个单一轴
Clean 类 classifier	Precision 约 0.88、Recall 约 0.91、F1 约 0.90	小型 encoder 足以规模化 teacher 信号
0.50 threshold	约减少 8% 数据	温和清洗
0.90 threshold	约减少 25% 数据	激进清洗
训练对比	两种 cleaned 版本均比原始数据更快/更高达到 HellaSwag 目标	行级清洗可提高 token efficiency

推荐架构 Stage 1: 文档级便宜规则砍掉显然坏的数据；Stage 2: 模型式文档评分分桶；Stage 3: 只对中高价值但局部污染的文档做 paragraph/line refinement。这样比“所有数据都做昂贵行级模型”更符合规模经济。

7. 安全、隐私与“有害内容”清洗：不要与一般质量混为一谈

Dolma 将风险缓解作为独立处理：使用正则识别并 mask 邮箱、电话号码和 IP 地址，同时用分类器检测 harmful/obscene 内容，并选择较高阈值以降低对非正式文本的误删。[3] 这是一种重要设计：风险分数应与 quality score 分离，因为“有风险”不等于“语言质量低”。

风险类型	常见策略	建议动作	为什么不应并入 quality
PII	regex/NER/classifier	优先 span-level mask / hash / drop 高风险文档	高质量文本也可能含 PII
成人/露骨内容	URL + classifier	依据政策分级过滤	语言可能流畅且知识密度高
仇恨/骚扰/毒性	classifier + context	单独 risk bucket；做下游 toxicity eval	定义高度语境化
秘密/凭据	pattern + entropy	代码/文本专用 secret scanner	属于泄露风险，不是文本质量
版权/许可元信息	来源/license tag	保留 provenance，按授权策略路由	法律治理维度独立

Longpre 等对 28 个 1.5B 模型的实验发现：质量/毒性过滤在标准 benchmark 与有毒生成风险之间存在 trade-off，没有一个适用于所有目标的统一过滤策略。[6] 因此 safety cleaning 必须明确优化目标，而不是默认“过滤越多越安全、模型也越强”。

另一个关键警告来自对 C4 的研究：基于 bad-word blacklist 的过滤会不成比例地删除与少数群体相关的文本。[7] 这说明词表式风险过滤必须做群体/方言/主题偏差审计。

Know-how 对风险类清洗建立三套独立指标：① detection recall/precision；② 对目标群体/语言的差异删除率；③ 训练后 toxicity/privacy eval。只看“删掉了多少”没有意义。

8. Decontamination：它是“评测完整性”问题，不是普通清洗

Web 语料可能包含 benchmark 题目、答案、教程解答或改写版本。Dolma、DCLM 等都把 decontamination 单独处理和报告。[2][3] 研究还指出，简单 n-gram overlap 对改写、模板化题目和 ground-truth contamination 并不完备。[13]

检测层次	方法	优点	盲点
Exact	hash / exact string	快、低误报	轻微改写即失效
n-gram	共享 n-gram / Bloom filter	可扩展	阈值敏感；常见短语误报
Fuzzy	MinHash / edit similarity	覆盖近重复	计算更大
Semantic	embedding similarity	可检测改写	高成本、阈值难校准
Answer-aware	题干 + 答案分别检测	捕获 ground-truth 泄露	需要 benchmark 结构信息

工程原则 Decontamination 最好产出 contamination report + match evidence + removal policy。尤其在公开研究中，不应只说“已去污染”，而应报告算法、阈值、被命中数据量和 benchmark 列表。

9. 清洗不是越狠越好：Quality × Quantity × Diversity × Horizon

这是现代预训练数据工程里最重要的结论之一。FineWeb/DCLM 的短中期实验表明，强质量过滤可以显著提高单位 token 的能力收益；但 Nemotron-CC 指出，FineWeb-Edu/DCLM 风格的激进模型过滤可能删除约 90% 数据，导致长 token horizon 下缺少足够独特的真实文本。[2][4]

训练场景	更看重什么	清洗策略倾向	主要风险
小模型/短 horizon	token efficiency	更强质量阈值	过拟合高质量风格
中等 horizon	质量+覆盖平衡	quality buckets + mixture	域失衡
超长 horizon	unique real tokens	保留更多中等质量数据，降低重复	噪声累积
继续预训练/领域化	目标域纯度	domain-specific filters	灾难性遗忘/分布窄化

Nemotron-CC 报告：其完整 6.3T-token 数据在 MMLU 上可匹配 DCLM，同时拥有约 4 倍的不同真实 token；其高质量子集则在 1T-token 训练下表现更强。[4] 这意味着“最佳清洗阈值”是训练预算的函数，而不是数据集的永久常数。

核心公式（概念） 最佳清洗策略 $\approx \operatorname{argmax} \operatorname{ModelUtility}(\operatorname{Filter}, \operatorname{TokenBudget}, \operatorname{ModelSize}, \operatorname{EvalMix}, \operatorname{RiskBudget})$ 。换句话说：过滤器不是数据预处理常量，而是训练系统的超参数。

10. Cleaning × Parsing × Dedup 的相互作用

很多数据管线优化失败，不是单个模块差，而是模块顺序改变了分布。例如 FineWeb 发现跨 96 个 Common Crawl snapshot 做全局 MinHash，会把旧 crawl 中大量“常见但较好”的页面删掉，反而使保留下来的尾部数据质量更差；按 snapshot 独立去重效果更好。[1] 这说明 dedup 会改变后续 quality 分布，cleaning 不能独立调参。

相互作用	典型错误	正确做法
Parsing → Cleaning	正文抽取失败后被 quality filter 删除	先记录 parser_health；修 parser 或路由二级解析
Cleaning → Dedup	先极强过滤导致重复结构比例变化	在 ablation 中同时看 keep-rate 与 duplication
Dedup → Cleaning	去重后长尾垃圾被相对上采样	去重前后分别统计 quality histogram
Language → Quality	英文 classifier 过滤多语数据	per-language / multilingual classifier
Safety → Coverage	bad-word 规则删少数群体文本	差异删除率审计 + context classifier
Cleaning → Mixing	只保留 top quality 后领域分布塌缩	保留 bucket/metadata，由 mixing 恢复目标覆盖

11. 如何证明一个清洗规则有效：Data Ablation 体系

FineWeb 和 DCLM 最值得复用的 know-how 是：把数据管线变成可实验的模型训练问题。只看人工抽检、PPL、classifier F1 都不足以证明“这批数据对最终 LLM 更好”。[1][2]

标准 data ablation

Fixed Raw Pool	→	Variant A/B	→	Equal Tokens	→	Same Model/H P	→	Train Proxy LM	→	Eval Suite	→	Decision
----------------	---	-------------	---	--------------	---	----------------	---	----------------	---	------------	---	----------

控制变量	要求	原因
模型架构	完全一致	隔离数据效果
训练 token	相同 token 数	避免“数据多”伪装成“数据好”
训练 steps/LR	一致	避免优化差异
Tokenizer	一致	避免分词效率改变
随机性	最好多 seed / 多随机数据子集	降低小模型评测噪声
评测集	低方差 + 多能力维度	单一 benchmark 会过拟合清洗目标
keep-rate	必须记录	收益要与数据损失一起看
cost	CPU/GPU-hours、吞吐	最终要算 end-to-end ROI

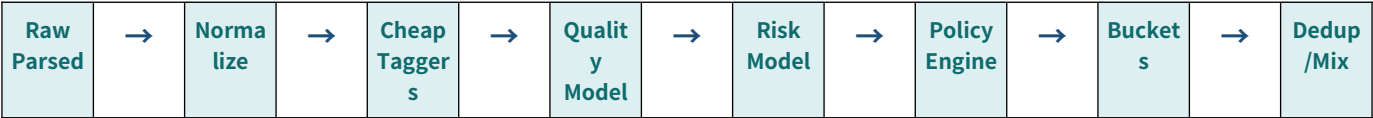
DCLM 进一步给出一个很实用的结果：400M/1B 等小规模实验对 7B 级数据策略排序具有较高相关性，因此可以用 proxy model 快速迭代清洗策略，再把少数候选放大。[2]

11.1 推荐的 Filter Evaluation Scorecard

维度	指标示例	必须回答的问题
Data utility	Core/MMLU/Reasoning/LM loss	单位 token 是否更有效?
Retention	docs/tokens keep rate	收益换来了多少数据损失?
Diversity	domain/lang/source entropy	是否把分布洗窄?
Uniqueness	unique tokens / dup rate	长 horizon 是否够用?
Risk	PII/toxicity/privacy eval	风险是否真的下降?
Bias	group/language differential removal	是否系统性误删某些群体?
Cost	docs/s、tokens/s、\$/T token	是否能在 PB 级运行?
Reproducibility	version/hash/config	能否完全重建?

12. Production Blueprint: 把 Cleaning 做成“标签系统 + 策略层”

综合工程推断 以下架构是对 FineWeb、DCLM、Dolma、RedPajama 等公开管线的综合工程抽象，不是某一篇文章原样提出的系统。



层	职责	典型实现	设计要点
0. Provenance	来源/URL/snapshot/license/parser	metadata schema	任何删除都可追溯
1. Normalize	低风险文本修复	Unicode/whitespace/control-char	不隐藏语义性删除
2. Cheap Tags	统计特征/语言/长度/重复	Rust/Beam/Spark map	尽量一次扫描产生多 tag
3. Quality Score	学习式质量排序	fastText/encoder	输出连续分数而非只有 bool
4. Risk Score	PII/toxicity/adult/secret	regex+classifier	与 quality 独立
5. Policy	阈值/组合/例外	versioned config	训练阶段可选择不同策略
6. Buckets	A/B/C + domain/lang	partitioned storage	为 mixing 提供控制面
7. Audit	采样审查/差异删除率	dashboard + manifests	持续发现分布漂移

12.1 为什么 Tag-first 优于 Delete-first

Delete-first	Tag-first
规则变化后必须重跑全部上游数据	可直接改变 policy / threshold
无法知道 “为什么被删”	reason code 可解释
无法做 counterfactual ablation	同一原始池可构造多个变体
难以支持短/长 horizon 不同策略	不同训练阶段可读取不同 bucket
偏差发现后难恢复	可重新纳入被误删样本

12.2 一个推荐 Canonical Cleaning Record

```
{
  "doc_id": "...",
  "source": {"url": "...", "snapshot": "...", "parser": "..."},
  "text": "...",
  "lang": {"label": "en", "score": 0.87, "mixed_ratio": 0.04},
  "quality": {"score": 0.82, "bucket": "B", "model": "qf-v7"},
  "heuristics": {"short_line_frac": 0.21, "dup_line_char_frac": 0.02},
  "risk": {"pii": 0.01, "toxicity": 0.03, "adult": 0.00},
  "actions": ["mask_email"],
  "policy_version": "clean-2026-08",
  "keep": true
}
```

13. 典型失败模式与诊断

失败模式	表面现象	真实原因	诊断方法	修复
Benchmark 上升但长训下降	短训很好，继续训练停滞	过滤太狠，unique token 不足	retention/unique-token curve	放宽中档 bucket，降低重复
Quality classifier 很准但模型没变强	F1 高，LM eval 无提升	标签定义与训练能力不一致	data ablation	重做正样本/目标能力
某语言/方言突然变差	多语 eval 下滑	LID/规则误杀	按 language/group removal rate	降低阈值/二级路由
解析错误被当成低质	大量 table/list 文档被删	parser 输出损坏	parser_health vs quality cross-tab	修 parser 后再评质量
去重后 quality 变差	保留集人工看更脏	dedup 上采样尾部垃圾	去重前后 score histogram	重新排序 dedup/cleaning
安全过滤导致群体偏差	特定主题覆盖下降	blocklist proxy 偏差	group/topic sampling audit	context model + policy exceptions
成本失控	PB 级跑不完	所有数据都上大模型	stage cost profiling	teacher→small classifier；级联过滤

14. 可直接落地的 Data Cleaning Checklist

- 把 Parsing、Cleaning、Dedup、Decontam、Mixing 的输入输出 schema 分开。
- Normalization 只做可解释、低风险转换；任何不可逆删除都要 reason code。
- Language ID 输出概率与混合语言信息，不只输出 keep/drop。
- 所有 heuristic 先统计分布，再设计阈值，再用 proxy LM ablation 验证。
- Quality classifier 的正样本选择必须与目标能力一致，并保留多领域/多风格覆盖。
- 同时保存连续 quality score 和离散 bucket，阈值作为 policy 版本化。
- 采用 document → paragraph/line 的级联，而不是所有数据直接做高成本细粒度模型。
- PII / toxicity / adult / secrets 与 quality 独立打分和治理。
- 对 safety/blocklist 做语言、群体、主题的 differential removal audit。
- Decontamination 独立报告：benchmark、算法、阈值、命中量、处理方式。
- 每个 filter variant 记录 docs/tokens keep-rate、domain/lang coverage、unique tokens 与成本。
- 至少用一个小模型做固定 token、固定超参的 A/B 预训练实验。
- 针对最终训练 horizon 重新调过滤强度：短训优先 token efficiency，长训关注 unique real tokens。
- 保留 rejected/quarantine manifest，允许未来重放和反事实实验。
- 把 pipeline config、模型权重、规则列表、阈值、代码 commit 一起版本化。

15. Know-Why / Know-How 总结表

主题	Know-why: 为什么	Know-how: 怎么做
规范化	减少表示噪声，不应改变语义	Unicode/控制字符/空白低风险处理 + 版本化
语言清洗	训练分布先由语言选择塑形	score + script + mixed tags; per-language calibration
启发式过滤	PB 级最便宜的第一道门	统计特征→阈值候选→小模型 ablation
模型质量过滤	复杂“有用性”很难靠规则表达	teacher/reference data→轻量 classifier→score buckets
细粒度清洗	局部脏不代表整篇垃圾	级联到 paragraph/line，仅处理高价值候选
安全/隐私	风险目标与能力目标不同	risk score 独立；mask/drop/policy 分层
去污染	保证评测可信而非提高文本质量	exact+n-gram+fuzzy/semantic 分层检测并报告
Horizon	过滤强度改变 unique token 预算	短训强过滤；长训保留更多中等质量独特数据
实验验证	人工“看起来更干净”不等于 LM 更强	固定模型/算力/token，训练 proxy LM A/B
生产系统	清洗规则会持续变化	Tag-first、reversible、policy-driven、observability

最终认知 LLM 预训练的数据清洗不是“垃圾识别算法”，而是一套数据选择与风险治理系统。真正成熟的管线不会只问“这条文本脏不脏”，而会问：它给模型带来什么学习价值？删掉它损失了什么覆盖？在当前训练 horizon 下值得保留吗？这个判断能否通过可复现实验得到支持？

参考资料与研究依据

以下以公开论文/官方开放数据项目为主。本文中的工程蓝图与部分设计建议属于对这些资料的综合推断，已在正文中标注。

- [1] Penedo et al., The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale, 2024. <https://arxiv.org/abs/2406.17557>
- [2] Li et al., DataComp-LM: In search of the next generation of training sets for language models, v4 2025. <https://arxiv.org/abs/2406.11794>
- [3] Soldaini et al., Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research, 2024. <https://arxiv.org/abs/2402.00159>
- [4] Su et al., Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset, v2 2025. <https://arxiv.org/abs/2412.02595>
- [5] Henriksson et al., FinerWeb-10BT: Refining Web Data with LLM-Based Line-Level Filtering, updated 2026. <https://arxiv.org/abs/2501.07314>
- [6] Longpre et al., A Pretrainer's Guide to Training Data: Measuring the Effects of Data Age, Domain Coverage, Quality, & Toxicity, 2023. <https://arxiv.org/abs/2305.13169>
- [7] Dodge et al., Documenting Large Webtext Corpora: A Case Study on the Colossal Clean Crawled Corpus, 2021. <https://arxiv.org/abs/2104.08758>
- [8] Lee et al., Deduplicating Training Data Makes Language Models Better, 2021. <https://arxiv.org/abs/2107.06499>
- [9] Kandpal et al., Deduplicating Training Data Mitigates Privacy Risks in Language Models, 2022. <https://arxiv.org/abs/2202.06539>
- [10] Weber et al., RedPajama: an Open Dataset for Training Large Language Models, 2024. <https://arxiv.org/abs/2411.12372>
- [11] Laurençon et al., The BigScience ROOTS Corpus: A 1.6TB Composite Multilingual Dataset, 2023. <https://arxiv.org/abs/2303.03915>
- [12] Messmer et al., Enhancing Multilingual LLM Pretraining with Model-Based Data Selection, 2025. <https://arxiv.org/abs/2502.10361>
- [13] Jiang et al., Investigating Data Contamination for Pre-training Language Models, 2024. <https://arxiv.org/abs/2401.06059>

附录 A：关键公开研究的“数据清洗贡献”对比

项目	年份	清洗贡献	最值得复用的结论
C4 / C4 audit	2020/2021	启发式规则、bad-word 过滤及后续偏差审计	规则简单有效，但 blocklist 可产生群体偏差 [7]
MassiveText/Gopher	2021	长度、停用词、重复字符等 heuristics	规则式质量过滤成为后续管线常见基线
Dolma	2023-2024	heuristics + lang + PII + toxicity + decontam	风险与质量要分层；公开 pipeline 与 ablations [3]
FineWeb	2024	50+ 统计特征、阈值消融、规则组合	用 proxy LM 而不是直觉决定规则 [1]
DCLM	2024-2025	系统比较多种模型式过滤	fastText + 好参考数据可胜过更复杂方法 [2]
RedPajama-V2	2024	原始数据+大量 quality signals/metadata	保留信号，让社区重新组合过滤策略 [10]
Nemotron-CC	2024-2025	长 horizon 下减少过度 filtering	数据纯度与 unique token 数量必须共同优化 [4]
FinerWeb	2025-2026	LLM teacher + 行级 encoder 清洗	细粒度 cleaning 可回收好正文并提升 token efficiency [5]

版本：v1.0 · 研究快照 2026-08-13 · 主题范围仅限 LLM 预训练数据清洗（Data Cleaning）。